IS1 - TLA$^+$

1 Introducción

Consideremos la especificación de un reloj que solo muestra la hora. La variable *hr* representa la hora que muestra el reloj. Debemos especificar todas las trazas posibles del reloj, por lo que necesitamos un predicado inicial que asegure que *hr* puede tomar cualquier valor entre 1 y 12. Definimos el predicado inicial como sigue:

$$HCini \triangleq hr \in (1..12) \tag{1}$$

También tendremos una fórmula *HCnxt* que nos permita relacionar los valores de un estado con los del siguiente, donde los valores del siguiente estado se indican primando las variables. En este caso definimos

$$HCnxt \triangleq hr' = \mathsf{IF}\ hr \neq 12\ \mathsf{THEN}\ hr + 1\ \mathsf{ELSE}\ 1 \tag{2}$$

Una fórmula en la que aparecen variables tanto primadas como no primadas se llama **acción**. Una acción es verdadera o falsa en un paso (transición).

Utilizamos el operador $\Box$ de la lógica temporal para indicar que una fórmula siempre es verdadera. En nuestro ejemplo, como cada paso debe satisfacer el predicado *HCnxt*, escribimos:

$$HCini \land \Box HCnxt \tag{3}$$

1.1 Stuttering steps

Los pasos en los que una variable de estado no se modifica (como $h' = h$) se llaman **stuttering steps**. En nuestro ejemplo la especificación puede tener que admitir estos pasos si el reloj tuviera otra variable (por ejemplo, la temperatura, cuyo cambio puede producirse sin que cambie la hora). Para que la fórmula admita stuttering steps escribimos

$$HCini \land \Box[HCnxt]_{hr} \tag{4}$$

donde $[HCnxt]_{hr}$ es lo mismo que $HCnxt \lor (hr' = hr)$.

Nuestra especificación (por ahora) del reloj es

$$HC \triangleq HCini \land \Box[HCnxt]_{hr} \tag{5}$$

1.2 Fórmulas temporales

La fórmula *HC* es una fórmula temporal. Una **fórmula temporal** es una afirmación sobre trazas (transiciones de estado). Decimos que una traza *satisface HC* sii *HC* es una afirmación verdadera sobre la traza.

Si el reloj se comporta apropiadamente, *hr* debería ser un entero entre 1 y 12 en cada estado de cualquier traza que satisfaga la especificación del reloj *HC*. La fórmula *HCini* afirma que *hr* es un entero entre 1 y 12, y $\Box HCini$ afirma que

HCini es siempre verdadero. Entonces, $\Box$*HCini* debería ser verdadero para cualquier traza que satisfaga *HC*. Toda traza debe satisfacer la fórmula

$$HC \Rightarrow \Box HCini \tag{6}$$

Un **teorema** es una fórmula temporal satisfecha por toda traza. Por ejemplo, (6) es un teorema.

1.3 Especificaciones en TLA$^+$

Las especificaciones en TLA$^+$ están particionadas en **módulos**.

La siguiente construcción inicia un módulo o submódulo llamado M utilizando la palabra MODULE.

MODULE M

Los operadores aritméticos como + no están en TLA$^+$ per sé, pero están definidos en *módulos*. Los operadores de números naturales se definen en el módulo *Naturals*.

Para incorporar declaraciones, definiciones, suposiciones y teoremas de los módulos llamados $M_1, \ldots, M_n$ en el módulo actual escribimos:

EXTENDS $M_1, \ldots, M_n$

Con la palabra VARIABLE podemos declarar a cada x_j como variables.

VARIABLE $x_1, \ldots, x_n$

Los operadores matemáticos ordinarios de la lógica y la teoría de conjuntos (como $\wedge$ y $\in$) están en TLA$^+$.

Los teoremas como el siguiente aseguran que P puede probarse a partir de las definiciones e hipótesis (suposiciones) del módulo actual. Si la fórmula P fuera falsa, el módulo sería incorrecto.

THEOREM P

A continuación, el módulo *HourClock* del ejemplo. La especificación del reloj es la definición de *HC*, incluyendo las definiciones de las fórmulas *HCnxt* y *HCini* y de los operadores .. (donde $i..j$ representa el conjunto de enteros entre i y j) y + que apaeecen en la definición de *HC*.

MODULE *HourClock*

EXTENDS *Naturals*
VARIABLE *hr*
$HCini \triangleq hr \in (1..12)$
$HCnxt \triangleq hr' = \text{IF } hr \neq 12 \text{ THEN } hr + 1 \text{ ELSE } 1$
$HC \triangleq HCini \wedge \Box[HCnxt]_{hr}$

THEOREM $HC \Rightarrow \Box HCini$

Una especificación alternativa es utilizando el operador de módulo de *Naturals*, que escribimos %. Podemos definir:

$$HCnxt2 \triangleq hr' = (hr\%12) + 1 \tag{7}$$
$$HC2 \triangleq HCini \wedge \Box[HCnxt2]_{hr} \tag{8}$$

donde *HC* y *HC*2 son equivalentes, es decir, $HC \equiv HC2$ es un teorema.

1.4 Otros constructores de módulos

En el ejemplo de abajo, se declara cada C_j como constante. En el último caso de la enumeración se indica que C es un operador que recibe la cantidad indicada de argumentos.

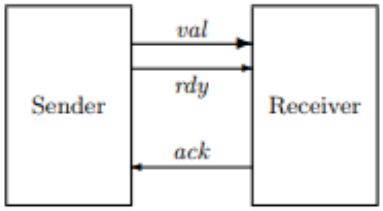
CONSTANTS $C_1, \ldots, C_n, C(_, \ldots, _)$

La siguiente construcción afirma que P es una hipótesis.

ASSUME P

2 Invariantes de tipo, récords y acciones

Consideremos ahora la especificación de una interfaz para transmisión de datos entre dispositivos asíncronos. Tenemos un *sender* y un *receiver* conectados como se muestra en la imagen.



Los datos se envían en *val*, y *rdy* y *ack* se utilizan para sincronización.

Asumimos que hay un conjunto *Data* de valores que pueden ser enviados como datos y lo declaramos como constante.

2.1 Invariante de tipo

Una **función de estado** es una expresión ordinaria (sin variables primadas ni $\Box$) que puede contener variables y constantes.

Un **predicado de estado** es una función de estado booleana.

Un **invariante** *Inv* de una especificación *Spec* es un predicado de estado tal que $Spec \Rightarrow \Box Inv$ es un teorema.

Una variable v tiene **tipo** T en una especificación *Spec* sii $v \in T$ es un invariante de *Spec*.

En nuestro ejemplo podemos definir un invariante de tipo como sigue:

$$\begin{aligned} TypeInvariant \triangleq\ & \wedge\ val \in Data \\ & \wedge\ rdy \in \{0,1\} \\ & \wedge\ ack \in \{0,1\} \end{aligned}$$

La fórmula *TypeInvariant* no aparecerá como parte de la especificación sino como teorema: la especificación debe implicar el invariante.

Cabe aclarar que TLA$^+$ es no tipado. La correctitud de tipos es solo una propiedad de invariancia.

2.2 Variables que no cambian

Para la transición al siguiente estado *Next* vamos a definir dos acciones: *Send* y *Rcv*. Un *Next*-paso es un $Send \vee Recv$-paso.

La acción *Send* está habilitada sii $rdy = ack$. Al relizar la transición, el nuevo valor val' puede ser cualquier elemento de *Data*, el valor de rdy' cambia el valor de *rdy* a 0 ó 1, y *ack* no se modifica, lo que indicamos con la palabra UNCHANGED (donde UNCHANGED v iguala $v' = v$).

Por otro lado, un *Rcv*-paso está habilitado sii $rdy \neq ack$. En este caso, se modifica el valor de *ack* y tanto *val* como *rdy* se mantienen. Utilizamos los símbolos $\langle$ y $\rangle$ para indicar todas las variables que no cambian.

Como la especificación debe admitir stuttering steps, definimos *Spec* como sigue:

$$Spec \triangleq Init \wedge \Box[Next]_{\langle val, rdy, ack \rangle}$$

A continuación la especificación completa:

MODULE *AsynchInterface*

EXTENDS *Naturals*
CONSTANT *Data*
VARIABLES *val*, *rdy*, *ack*

$$\begin{aligned}
\mathit{TypeInvariant} \triangleq\ & \land\ \mathit{val} \in \mathit{Data} \\
& \land\ \mathit{rdy} \in \{0,1\} \\
& \land\ \mathit{ack} \in \{0,1\}
\end{aligned}$$

$$\begin{aligned}
\mathit{Init} \triangleq\ & \land\ \mathit{val} \in \mathit{Data} \\
& \land\ \mathit{rdy} \in \{0,1\} \\
& \land\ \mathit{ack} = \mathit{rdy} \\
\mathit{Send} \triangleq\ & \land\ \mathit{rdy} = \mathit{ack} \\
& \land\ \mathit{val} \in \mathit{Data} \\
& \land\ \mathit{rdy}' = 1 - \mathit{rdy} \\
& \land\ \text{UNCHANGED } \mathit{ack} \\
\mathit{Rcv} \triangleq\ & \land\ \mathit{rdy} \neq \mathit{ack} \\
& \land\ \mathit{ack}' = 1 - \mathit{ack} \\
& \land\ \text{UNCHANGED } \langle \mathit{val}, \mathit{rdy} \rangle \\
\mathit{Next} \triangleq\ & \mathit{Send} \lor \mathit{Rcv} \\
\mathit{Spec} \triangleq\ & \mathit{Init} \land \Box[\mathit{Next}]_{\langle \mathit{val},\mathit{rdy},\mathit{ack} \rangle}
\end{aligned}$$

THEOREM $\mathit{Spec} \Rightarrow \Box\, \mathit{TypeInvariant}$

2.3 Records y acciones parametrizadas

El primer problema de nuestra especificación es que usa tres variables para describir una sola interfaz. Un sistema podría usar varias instancias de esta interfaz. Reemplazamos las tres variables *val*, *rdy* y *ack* por una sola variable *chan*, que será un **record**.

Ahora tendremos como invariante de tipo un conjunto de records dado por $[\mathit{val} : \mathit{Data}, \mathit{rdy} : \{0,1\}, \mathit{ack} : \{0,1\}]$. Los records no están ordenados y si el record es r, accedemos al campo c con $r.c$.

Inicialmente, *chan* puede tomar cualquier valor del conjunto de records definido en los que $\mathit{chan.ack} = \mathit{chan.rdy}$.

Definimos la acción $\mathit{Send}(d)$ que envía un valor d. Como la transición al siguiente estado está dada por un $\mathit{Send}(d)$-paso (para algún d en *Data*) o un *Rcv*-paso, definimos:

$$\mathit{Next} \triangleq (\exists\, d \in \mathit{Data} : \mathit{Send}(d)) \lor \mathit{Rcv}$$

La acción $\mathit{Send}(d)$ asegura que chan' iguala al record r tal que $r.\mathit{val} = d$, $r.\mathit{rdy} = 1 - \mathit{chan.rdy}$ y $r.\mathit{ack} = \mathit{chan.ack}$. En TLA$^+$ escribimos esto como:

$$[\mathit{val} \mapsto d,\ \mathit{rdy} \mapsto 1 - \mathit{chan.rdy},\ \mathit{ack} \mapsto \mathit{chan.ack}]$$

Una alternativa es decir que chan' es el record *chan* con los campos *val* y *rdy* modificados utilizando EXCEPT de la siguiente manera:

$$[\mathit{chan}\ \text{EXCEPT}\ !.\mathit{val} = d,\ !.\mathit{rdy} = 1 - @]$$

El símbolo ! representa al nuevo record creado por EXCEPT al modificar *chan*, y @ representa *chan.rdy*.

La especificación completa es la siguiente:

MODULE *Channel*

EXTENDS *Naturals*
CONSTANT *Data*
VARIABLE *chan*
$TypeInvariant \triangleq chan \in [val : Data, rdy : \{0, 1\}, ack : \{0, 1\}]$

$Init \triangleq \land TypeInvariant$
$\quad \land chan.ack = chan.rdy$
$Send \triangleq \land chan.rdy = chan.ack$
$\quad \land chan' = [chan \,\text{EXCEPT}\, !.val = d, !.rdy = 1 - @]$
$Rcv \triangleq \land chan.rdy \neq chan.ack$
$\quad \land chan' = [chan \,\text{EXCEPT}\, !.ack = 1 - @]$
$Next \triangleq (\exists\, d \in Data : Send(d)) \lor Rcv$
$Spec \triangleq Init \land \Box[Next]_{chan}$

THEOREM $Spec \Rightarrow \Box TypeInvariant$

2.4 Definiciones

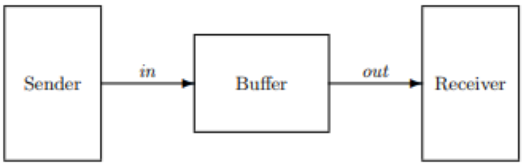
Todos los símbolos (identificadores y operadores) tienen un *scope*. El scope de declaraciones CONSTANT y VARIABLE es la parte del módulo que les sucede. Cuando escribimos EXTENDS *Naturals* estamos extendiendo el scope de los símbolos definidos en el módulo *Naturals* al módulo actual.

En una declaración de la forma $\exists\, v \in S : exp$, el scope de la variable v es exp.

Un símbolo no puede ser declarado o definido si ya tiene un significado.

3 Instancias y substitución

El siguiente ejemplo será un buffer FIFO como ilustra la imagen, donde *in* y *out* son dos canales de comunicación con el buffer. Usaremos el módulo *Channel*.



3.1 Secuencias

El módulo *Sequences* define operaciones en secuencias finitas. Representamos secuencias finitas como tuplas $\langle 3, 2, 1\rangle$. El módulo define los siguientes operadores:

$Seq(S)$	Es el conjunto de todas las secuencias de elementos del conjunto S ($\langle 3, 7\rangle$ es un elemento de $Seq(Nat)$).
$Head(s)$	El primer elemento de la secuencia s.
$Tail(s)$	La cola de la secuencia s, es decir, sin su primer elemento.
$Append(s, e)$	La secuencia obtenida de appendear el elemento e al final de la secuencia s.
$s \circ t$	La secuencia obtenida de concatenar las secuencias s y t.
$Len(s)$	La longitud de la secuencia s.
$\langle\rangle$	La secuencia vacía.

3.2 Instancias

La especificación tendrá tres variables: *in*, *out* y q, que representa la secuencia de mensajes en el buffer, es decir, aquellos mensajes que fueron enviador por *sender* pero aún no fueron recibidos por *receiver*.

Queremos dos instancias distintas de *Channel*: una para el canal *in* y otra para el canal *out*. El conjunto de datos (la constante *Data*) en *Channel* será la constante *Message*, y la variable *chan* será *in*. Definimos la instancia como sigue:

$InChan \triangleq$ INSTANCE *Channel* WITH $Data \leftarrow Message,\ chan \leftarrow in$

Para cada símbolo σ definido en el módulo *Channel*, esto define $InChan!\sigma$ con el mismo significado en este módulo que el que σ tenía en *Channel*, excepto por las substituciones que siguen a WITH.

Las inveriantes de tipo de *in* y *out* vendrán del módulo *Channel*, y debemos sumar la propia de nuestro módulo: $q \in Seq(Message)$.

Tenemos cuatro posibles acciones para pasar a un siguiente estado:

$SSend(msg)$	El sender envía el mensaje *msg* al canal *in*.
$BufRcv$	El buffer recive el mensaje del canal *in* y lo appendea a q.
$BufSend$	El buffer elimina el tope de q y lo envía por el canal *out*.
$RRcv$	El receiver recibe el mensaje del canal *out*.

La especificación es la siguiente.

MODULE *InnerFIFO*

EXTENDS *Naturals*, *Sequences*
CONSTANT *Message*
VARIABLES in, out, q
$InChan \triangleq$ INSTANCE *Channel* WITH $Data \leftarrow Message,\ chan \leftarrow in$
$OutChan \triangleq$ INSTANCE *Channel* WITH $Data \leftarrow Message,\ chan \leftarrow out$

$$
\begin{aligned}
Init \triangleq\ & \land\ InChan!Init \\
& \land\ OutChan!Init \\
& \land\ q = \langle\rangle \\
TypeInvariant \triangleq\ & \land\ InChan!TypeInvariant \\
& \land\ OutChan!TypeInvariant \\
& \land\ q \in Seq(Message) \\
SSend(d) \triangleq\ & \land\ InChan!Send(msg) \\
& \land\ \text{UNCHANGED } \langle out,\ q\rangle \\
BufRcv \triangleq\ & \land\ InChan!Rcv \\
& \land\ q' = Append(q, in.val) \\
& \land\ \text{UNCHANGED } out \\
BufSend \triangleq\ & \land\ q \neq \langle\rangle \\
& \land\ OutChan!Send(Head(q)) \\
& \land\ q' = Tail(q) \\
& \land\ \text{UNCHANGED } in \\
RRcv \triangleq\ & \land\ OutChan!rcv \\
& \land\ \text{UNCHANGED } \langle in,\ q\rangle \\
Next \triangleq\ & \lor\ \exists\, msg \in Message : SSend(msg) \\
& \lor\ BufRcv \\
& \lor\ BufSend \\
& \lor\ RRcv \\
Spec \triangleq\ & Init \land \Box[Next]_{\langle in,\ out,\ q\rangle}
\end{aligned}
$$

THEOREM $Spec \Rightarrow \Box\, TypeInvariant$

3.2.1 Substitución

Consideremos la siguiente declaración:

$IM \triangleq$ INSTANCE M WITH $p_1 \leftarrow e_1, \ldots, p_n \leftarrow e_n$

Sea Σ un símbolo definido en el módulo M y sea d su definición "real". La declaración INSTANCE define la definición real de $IM!\Sigma$ como la expresión obtenida de d al reemplazar todas las instancias de p_i por la expresión e_i para cada i. La definición de $IM!\Sigma$ debe contener sólo los parámetros del módulo actual, no los del módulo M.

3.2.2 Instanciación parametrizada

Podemos reemplazar las dos instancias de *Channel* por una sola parametrizada:

$Chan(ch) \triangleq$ INSTANCE *Channel* WITH $Data \leftarrow Message$, $chan \leftarrow ch$

3.2.3 Substituciones implícitas

Cuando creamos una instancia donde las variables de ésta se substituyen por otras con el mismo nombre, no es necesario incluir esta substitución después del WITH. Por ejemplo, si en lugar de *Message* tuviéramos *Data*, no hace falta escribir que reemplazamos *Data* por *Data*:

$InChan \triangleq$ INSTANCE *Channel* WITH $chan \leftarrow in$

3.3 Ocultación de variables

La especificación *Spec* dada más arriba muestra variables internas como q, que deberían ser ocultadas. En TLA ocultamos variables con el cuantificador existencial $\exists$ de lógica temporal. La fórmula $\exists x : F$ es verdadera para una traza sii existe alguna secuencia de valores (una en cada estado de la traza) que pueda ser asignada a la variable x que haga verdadera la fórmula F.

No podemos escribir la fórmula $\exists q : Spec$ en *InnerFIFO* puesto que q ya está declarada allí. En cambio, definimos un nuevo módulo con una instancia parametrizada del módulo *InnerFIFO*.

MODULE *FIFO*

CONSTANT *Message*
VARIABLES *in*, *out*
$Inner(q) \triangleq$ INSTANCE *InnerFIFO*
$Spec \triangleq \exists q : Inner(q)!Spec$

3.4 Suposiciones (hipótesis)

Supongamos que queremos establecer un límite N de mensajes para el buffer. La única modificación es que la acción *BufRcv* no debería estar habilitada a no ser que haya menos de N mensajes en el buffer, es decir, que $Len(q) < N$.

La acción *BNext* en *InnerFIFO* debería ser igual a $Next \wedge (BufRcv \Rightarrow (Len(q) < N))$. Para poder describir N (tras declararla constante) escribimos la siguiente declaración:

ASSUME $(N \in Nat) \wedge (N > 0)$

que afirma que, en este módulo, asumimos que N es un número natural. Puede considerarse como hipótesis a la hora de probar cualquier teorema del módulo. Un módulo afirma que sus hipótesis implican sus teoremas. Las fórmulas utilizadas en una declaración ASSUME deben ser constantes: no pueden contener ninguna variable.

MODULE *BoundedFIFO*

EXTENDS *Naturals*, *Sequences*
VARIABLES *in*, *out*
CONSTANTS *Message*, *N*
ASSUME $(N \in Nat) \land (N > 0)$
$Inner(q) \triangleq$ INSTANCE *InnerFIFO*
$BNext(q) \triangleq \land Inner(q)!Next$
$\land Inner(q)!BufRcv \Rightarrow (Len(q) < N)$
$Spec \triangleq \exists q : Inner(q)!Init \land \Box[BNext(q)]_{\langle in,\, out,\, q\rangle}$

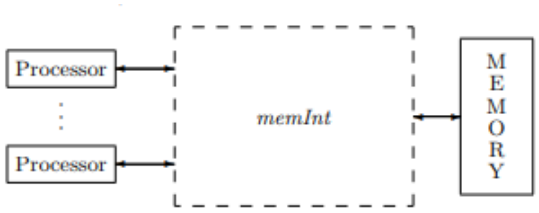
Debemos asumir que N es un natural pero no que *Message* es un conjunto. Esto se debe a que en TLA$^+$ todo valor es un conjunto (no necesariamente finito).

3.5 Especificaciones de sistema cerrado y abierto

Una fórmula como *Spec*, que describe el comportamiento correcto del sistema y de su entorno, se llama **especificación de sistema cerrado** o completo. Una **especificación de sistema abierto** es aquella que describe sólo el comportamiento correcto del sistema.

4 Funciones, recursión y subespecificación

Ahora veremos el ejemplo de una interfaz para una memoria. Llamaremos a un paso *Send* si un procesador envía un valor a la memoria y *Reply* si es la memoria quien envía a un proceso.



4.1 Subespecificación y booleanos

Representamos el estado de la interfaz con la variable *memInt*. Un *Send*-paso modifica el valor de dicha variable de alguna forma que no conocemos exactamente. Para subespecificar algo lo hacemos un *parámetro*. $Send(p, d)$ describe cómo *memInt* es modificado por un paso que representa que el procesador p envía el valor d a memoria. Sin embrago, TLA$^+$ sólo provee parámetros para constantes y variables, entonces declaramos *Send* como un operador constante $Send(p, d, memInt, memInt')$.

Declaramos *Send* como un operador constante que toma cuatro argumentos escribiendo

CONSTANT $Send(_, _, _, _)$

Si queremos asegurar que el valor de *Send* para cuatro parámetros cualquiera es un booleano utilizamos un ASSERT.

ASSUME $\forall\, p,\, d,\, miOld,\, miNew : Send(p,\, d,\, miOld,\, miNew) \in$ BOOLEAN

El símbolo BOOLEAN denota el conjunto {TRUE, FALSE}.

Además definiremos *InitMemInt* (el conjunto de posibles valores iniciales de *memInt*), *Proc* (el conjunto de procesadores), *Adr* (el conjunto de direcciones de memoria) y *Val* (el conjunto de valores que pueden ser enviados a memoria).

Representamos una solicitud de un procesador a la memoria como un *record* con un campo *op* que especifica el tipo de solicitud y campos adicionales que especifican sus argumentos. El conjunto de todas las solicitudes de lectura es

$[op : \{\text{"Rd"}\}, adr : Adr]$, mientras que el de todas las solicitudes de escritura será $[op : \{\text{"Wr"}\}, adr : Adr, val : Val]$. Definimos *MReq*, el conjunto de todas las solicitudes, como la unión de ambos.

Cuando la solicitud de lectura fall (no se halla un valor en la dirección especificada) debemos devolver algo. Para ello utilizamos CHOOSE. La expresión CHOOSE $x : F$ iguala algún valor arbitrario x que satisface la fórmula F. En nuestro caso definiremos *NoVal* de esa forma.

MODULE *MemoryInterface*

VARIABLE *memInt*
CONSTANTS *Message*
CONSTANTS $Send(_, _, _, _)$,
$Reply(_, _, _, _)$,
InitMemInt,
Proc,
Adr,
Val
ASSUME $\forall\, p,\, d,\, miOld,\, miNew \;\wedge Send(p,\, d,\, miOld,\, miNew) \in \text{BOOLEAN}$
$\wedge\, Reply(p,\, d,\, miOld,\, miNew) \in \text{BOOLEAN}$

$MReq \triangleq [op : \{\text{"Rd"}\}, adr : Adr] \cup [op : \{\text{"Wr"}\}, adr : Adr, val : Val]$
$NoVal \triangleq \text{CHOOSE } v : v \notin Val$

4.2 Funciones

Una **función** f tiene *dominio* DOMAIN f y asigna a cada elemento x de su dominio el valor $f[x]$. Dos funciones f y g son iguales ssi tienen el mismo dominio y $f[x] = g[x]\,\forall\, x$ en su dominio.

El *rango* de una función f es el conjunto de todos los valores de la forma $f[x]$ con x en el dominio de f. Para dos conjuntos cualesquiera S y T, el conjunto de todas las funciones cuyo dominio sea S y su rango sea cualquier subconjunto de T se escribe $[S \to T]$.

TLA$^+$ define $[x \in S \mapsto e]$ como la función f con dominio S tal que $f[x] = e\,\forall\, x \in S$. Por ejemplo, podemos definir $succ \triangleq [n \in Nat \mapsto n + 1]$.

Un *record* es una función cuyo dominio es un conjunto finito de cadenas de caracteres. Su constructor EXCEPT es un caso especial del mismo constructor para funciones, donde $!.c$ es una abreviación para $![\text{"c"}]$.

En general, $[f \text{ EXCEPT } ![c_1] = e_1, \ldots, ![c_n] = e_n]$ es la función $\hat{f}$ que es la misma que f excepto por $\hat{f}[c_i] = e_i$ para cada i. La expresión e_i puede contener el símbolo @, donde significa $f[c_i]$.

En el ejemplo de la memoria podemos agregar funciones como *mem*, que representa el valor de la memoria en cada dirección, *ctl*, que describe el estado de la solicitud de cada procesador, y *buf*, que contiene para cada procesador la solicitud o la respuesta. Debajo se ilustra un ejemplo.

$$\begin{bmatrix} ctl[p] &= \text{“rdy”} \\ buf[p] &= \cdots \\ mem[a] &= \cdots \end{bmatrix} \xrightarrow{Req(p)} \begin{bmatrix} ctl[p] &= \text{“busy”} \\ buf[p] &= req \\ mem[a] &= \cdots \end{bmatrix}$$

$$\xrightarrow{Do(p)} \begin{bmatrix} ctl[p] &= \text{“done”} \\ buf[p] &= NoVal \\ mem[a] &= v \end{bmatrix} \xrightarrow{Rsp(p)} \begin{bmatrix} ctl[p] &= \text{“rdy”} \\ buf[p] &= NoVal \\ mem[a] &= v \end{bmatrix}$$

4.3 Tuplas

Las **tuplas** en TLA$^+$ son en realidad *funciones*. La tupla $\langle a,\, b,\, c\rangle$ es la función con dominio $\{1, 2, 3\}$ que mapea 1 a a, 2 a b y 3 a c. De esta forma, $\langle a,\, b,\, c\rangle[2] = b$.

TLA$^+$ provee el operador de producto cartesiano $\times$, donde $A \times B \times C$ es el conjunto de todas las 3-uplas $\langle a,\, b,\, c\rangle$ tales que $a \in A$, $b \in B$ y $c \in C$.

El modulo *Sequences* define las secuencias finitas como tuplas. Una secuencia de longitud n es una función con dominio $1..n$. Más aún, s es una secuencia sii es igual a la función $[i \in 1..Len(s) \mapsto s[i]]$.

4.4 Definición de funciones recursivas

Es ilegal definir una función con una ocurrencia de la misma a la derecha de $\triangleq$. Una función es definida sólo después de su definición.

Una definición de la forma $f[x \in S] \triangleq e$ puede ser utilizada para definir recursivamente una función f con dominio S.

Por ejemplo, podemos definir la función factorial *fact* como sigue:

$$fact[n \in Nat] \triangleq \mathsf{IF}\ n = 0\ \mathsf{THEN}\ 1\ \mathsf{ELSE}\ n * fact[n-1]$$

4.5 Variables locales

La cláusula LET consiste de una secuencia de definiciones cuyo scope se extiende hasta el final de la cláusula IN.

Una definición LET puede tener parámetros. Estas definiciones locales pueden ser utilizadas para acortar una expresión reemplazando subexpresiones comunes con un operador.

Un LET puede aparecer como una subexpresión de una expresión IN.

La forma de utilizarlo es la siguiente:

$$\begin{array}{l} Def \triangleq \\ \quad \mathsf{LET}\ \ x \triangleq e \\ \qquad\quad\ y \triangleq f \\ \quad \mathsf{IN}\ \ \ x + y \end{array}$$

4.6 Invariancia

Un predicado de estado $P \wedge Q$ es siempre verdadero sii tanto P como Q son siempre verdaderos, entonces $\Box(P \wedge Q)$ es equivalente a $\Box P \wedge \Box Q$.

En general, cuando $P \wedge N \Rightarrow P'$ se cumple, decimos que el predicado P es un **invariante** de la acción N.

4.7 Implementación

En TLA, implementación es implicancia. Un sistema descrito por una fórmula *Sys* implementa una especificación *Spec* sii $Sys \Rightarrow Spec$ es un teorema.

5 Matemática en TLA$^+$

5.1 Conjuntos

Dos operadores poderosos de la teoría de conjuntos son los operadores unarios UNION y SUBSET:

UNION S	La unión de todos los elementos de S. Un valor e es un elemento de UNION S sii es un elemento de un elemento de S. Por ejemplo, UNION $\{\{1, 2\}, \{2, 3\}, \{3, 4\}\} = \{1, 2, 3, 4\}$.
SUBSET S	El conjunto de todos los subconjuntos de S (es decir, el conjunto de partes de S).

En TLA$^+$ podemos escribir conjuntos por *comprensión* de dos maneras:

$\{x \in S : p\}$	El subconjunto de S que consiste de todos los elementos x que satisfacen la propiedad p.
$\{e : x \in S\}$	El conjunto de elementos de la forma e para todo x en S.

Existe un módulo estándar *FiniteSets* que define dos operadores:

$Cardinality(S)$	El número de elementos en S si es un conjunto finito.
$IsFiniteSet(S)$	Verdadero sii S es un conjunto finito.

Una colección $\mathcal{C}$ es muy grande para ser un conjunto si es tan grande como una colección de conjuntos (es decir, si podemos asignar un elemento de $\mathcal{C}$ a todo conjunto). Esto es, $\mathcal{C}$ es muy grande para ser un conjunto si podemos definir un operador *SMap* tal que:

- $SMap(S)$ pertenece a $\mathcal{C}$ para cualquier conjunto S.
- Si S y T son dos conjuntos diferentes, entonces $SMap(S) \neq SMap(T)$.

5.2 Recursión

Utilizando el operador CHOOSE podemos definir la función *fact* de la siguiente manera:

$$fact \triangleq \mathsf{CHOOSE}\ fact :$$
$$\quad fact = [n \in Nat \mapsto \mathsf{IF}\ n = 0\ \mathsf{THEN}\ 1\ \mathsf{ELSE}\ n * fact[n-1]]$$

En general, $f[x \in S] \triangleq e$ es una abreviación para $f \triangleq \mathsf{CHOOSE}\ f : f[x \in S \mapsto e]$.

TLA$^+$ no permite definiciones circulares en las que dos o más funciones son definidas en términos de otra, con lo cuál no permite la recursión mutua. Por ejemplo, no pdoemos definir estas funciones:

$$f[n \in Nat] \triangleq \mathsf{IF}\ n = 0\ \mathsf{THEN}\ 17\ \mathsf{ELSE}\ f[n-1] * g[n]$$
$$g[n \in Nat] \triangleq \mathsf{IF}\ n = 0\ \mathsf{THEN}\ 42\ \mathsf{ELSE}\ f[n-1] + g[n-1]$$

Sin embargo, podemos definir dos funciones como campos de un record y luego definir cada función en términos del record.

$$mr[n \in Nat] \triangleq$$
$$\quad [f \mapsto \mathsf{IF}\ n = 0\ \mathsf{THEN}\ 17\ \mathsf{ELSE}\ mr[n-1].f * mr[n].g,$$
$$\quad g \mapsto \mathsf{IF}\ n = 0\ \mathsf{THEN}\ 42\ \mathsf{ELSE}\ mr[n-1].f + mr[n-1].g]$$
$$f[n \in Nat] \triangleq mr[n].f$$
$$g[n \in Nat] \triangleq mr[n].g$$

5.3 Funciones y operadores

Una *función* como *fact* es una expresión completa que denota un valor, mientras que un *operador* como *Tail* no lo es. Tanto $fact[n] \in S$ como $fact \in S$ son expresiones válidas, pero $Tail \in S$ no es sintácticamente válida (aunque $Tail(n) \in S$ sí). Otra diferencia es que los operadores no tienen dominio.

A diferencia de las funciones, los operadores no pueden definirse recursivamente. Normalmente podemos transformar una definición recursiva ilegal de un operador en una no recursiva usando una función recursiva.

Otra diferencia es que un operador puede tomar otro operador como argumento.

5.4 Operador CHOOSE

El uso más común de CHOOSE es para "nombrar" un valor único especificado. Por ejemplo, el módulo *Reals* define la division en el conjunto *Real* como:

$$a/b \triangleq \mathsf{CHOOSE}\ c \in Real : a = b * c$$

donde la expresión CHOOSE $x \in S : p$ significa CHOOSE $x : (x \in S) \wedge p$.

El operador CHOOSE es *determinista*: siempre elige el mismo valor.

6 Atomicidad y especificación

6.1 Composición

Si A y B son acciones, la acción $A \cdot B$ ejecuta primero A y luego B como un único paso. Es la acción definida al permitir que $s \to t$ sea un $A \cdot B$-paso sii existe un estado u tal que $s \to u$ es un A-paso y $u \to t$ es un B-paso.

Sean $S1$ y $S2$ especificaciones. Podemos transformar cualquier traza σ que satisface $S1$ en una traza $hat\sigma$ que satisfaga $S2$ reemplazando cada paso $s \overset{R \cdot L}{\to} t$ por el par de pasos $s \overset{R}{\to} u \overset{L}{\to} t$, para algún estado u. La especificación $S1$ requiere que cada R-paso sea sucedido inmediatamente por un L-paso, mientras que $S2$ admite trazas en las que otros pasos pueden aparecer entre R y L-pasos.

Dos acciones A y B *conmutan* sii $A \cdot B$ es equivalente a $B \cdot A$. Dos acciones conmutan si (i) cada una no modifica ninguna variable cuyo valor pueda ser modificado por la otra (ii) ninguna habilita o deshabilita la otra.

6.2 Forma de una especificación

En TLA las especificaciones tienen una forma sintáctica restringida: $Init \wedge \Box[Next]_{vars} \wedge Fairness_{vars}$, donde *Next* es por lo general la disyunción de las acciones permitidas en el sistema, *vars* son todas las variables del sistema, y *Fairness* es una conjunción numerable de fórmulas de *WF* y/o *SF*.

$Fairness_{vars}$ debe tener una de las siguientes formas:

- $WF_{vars}(A) \triangleq \Box(\Box enabled\langle A\rangle_{vars} \Rightarrow \Diamond\langle A\rangle_{vars})$
- $SF_{vars}(A) \triangleq \Box\Diamond enabled\langle A\rangle_{vars} \Rightarrow \Box\Diamond\langle A\rangle_{vars}$

donde A es una *subacción* de *Next*, es decir que implica *Next*, y $\Diamond\langle A\rangle_{vars}$ es igual a $\neg\,\Box[\neg\, A]_{vars}$.

Si $A_1, \ldots, A_n$ son acciones tales que, para $i \neq j$, siempre que la acción A_i está habilitada, A_j no puede habilitarse hasta que se dé un A_i-paso, entonces $WF_v(A_1) \wedge \cdots \wedge WF_v(A_n)$ es equivalente a $WF_v(A_1 \vee \cdots \vee A_n)$. Hay una regla igual para *SF*.

Según Lamport, *SF* y *WF* son equivalentes para una acción dada si siempre que esta está habilitada sólo puede deshabilitarse al ser ejecutada, y siempre debe preferirse la equidad débil por sobre la fuerte si no hay un motivo importante por el cual la segunda opción sea mejor.

6.3 Escribir la especificación

- Primero elegir las variables y definir el invariante de tipo y el predicado inicial.
- Luego escribir las acciones de transición de estado.
- Continuar con la parte temporal de la especificación. Si se quiere especificar propiedades liveness se debe elegir condiciones de fairness.
- Finalmente se pueden afirmar teoremas sobre la especificación o un teorema de correctitud de tipos.

7 Temporizadores

Especificaremos conficiones temporales por medio de temporizadores. A cada condición temporal se le asociará un temporizador. Cuando deba controlarse el paso de un cierto período de tiempo se iniciará el temporizador respectivo y se esperará a que este comunique el paso del tiempo estipulado mediante la señal *Timeout*. Si antes de la llegada de *Timeout* aparece un evento que hace innecesario seguir esperándola, entonces el temporizador puede deternerse.

Modelamos el tiempo real mediante la variable *now* que cuenta el tiempo desde cieeto momento inicial. El tiempo real avanza discreta pero indefinidamente cada vez que se "ejecuta" la acción *Tick*.

El tiempo real avanza una unidad de tiempo $\approx$ *Tick*

El temporizador "percibe" el paso del tiempo extendiendo el módulo *DK_RealTime*

Se configura el temporizador para que cuente el tiempo hasta t unidades de tiempo después de haber sido iniciado $\approx Set(t)$

Se inicia el temporizador $\approx$ *Start*

Se detiene el temporizador $\approx$ *Stop*

El temporizador alcanza la cota de tiempo estipulada $\approx$ *Timeout*

RTBound es un predicado definido en el módulo *RealTime* de la biblioteca estándar de TLA$^+$ que se usa para especificar restricciones temporales sobre acciones. La signatura del predicado es $RTBound(A,\ v,\ \delta,\ \epsilon)$ donde A es una acción, v es una o más variables de estado y δ y ϵ son números no negativos. Este predicado estipula que se *puede* dar un paso $\langle A\rangle_v$ sólo después de que $\langle A\rangle_v$ haya estado continuamente habilitada por δ unidades de tiempo desde el último $\langle A\rangle_v$-paso (o desde el inicio de la ejecución) y que se *debe* ejecutar antes de que $\langle A\rangle_v$ haya estado continuamente habilitada por más de ϵ unidades de tiempo desde el último paso (o desde el inicio de la ejecución).

En otras palabras, *RTBound* dice que A debe ejecutarse dentro del intervalo $[\delta, \epsilon]$ tomado desde la última ejecución de A siempre y cuando haya estado habilitada todo ese tiempo. En particular, si δ es 0 entonces el sistema tiene ϵ unidades de tiempo para ejecutar la acción desde el momento en que queda habilitada. En este caso el predicado *RTBound* se usa para establecer una cota superior para el tiempo que puede tomarse el sistema para ejecutar la acción.

MODULE *Timer*

EXTENDS *Naturals*, *DK_RealTime*, *RealTime*
VARIABLES *time*, *running*, *limit*
$TTypeInv \triangleq time,\ limit \in Nat \land running \in \{\text{"yes"},\ \text{"no"}\}$
$\langle limit,\ time,\ running\rangle$
$\langle limit,\ time,\ running,\ now\rangle$

$TInit \triangleq limit = 0 \land running = \text{"no"}$
$Set(l) \triangleq \land l > 0$
$\land running = \text{"no"}$
$\land limit' = l$
$\land$ UNCHANGED $\langle time,\ now,\ running\rangle$
$Start \triangleq \land running = \text{"no"}$
$\land limit > 0$
$\land time' = now$
$\land running' = \text{"yes"}$
$\land$ UNCHANGED $\langle now,\ limit\rangle$
$Timeout \triangleq \land running = \text{"yes"}$
$\land now - time \geq limit$
$\land running' = \text{"no"}$
$\land$ UNCHANGED $\langle now,\ time,\ limit\rangle$
$Stop \triangleq \land running = \text{"yes"}$
$\land running' = \text{"no"}$
$\land$ UNCHANGED $\langle now,\ time,\ limit\rangle$
$TNext \triangleq Start \lor Stop \lor Timeout \lor (\exists\, t \in Nat : Set(t))$
$TSpec \triangleq TInit \land \Box[TNext]_{sv} \land RTBound(Timeout,\ av,\ 0,\ 1)$

THEOREM $TSpec \Rightarrow \Box\, TTypeInv$

No incluimos *Tick* en *TNext* ya que metodológicamente *Tick* no es una acción del temporizador; el tiempo real avanza o no independientemente de la existencia del temporizador.

7.1 Arreglo de temporizadores

A continuación especificamos un módulo desde el cual se pueden manejar varios temporizadores simultáneamente.

MODULE *Timers*

EXTENDS *Naturals*, *DK_RealTime*, *RealTime*
VARIABLES *timers*
$TTypeInv \triangleq timers \in [Nat \to [t,\ l : Nat,\ r \in \{\text{"yes"},\ \text{"no"}\}]]$

$TInit \triangleq timers = [n \in Nat \mapsto [t \mapsto now,\ l \mapsto 0, r \mapsto \text{"no"}]]$

$Set(i,\ lim) \triangleq \land\ l > 0$
$\land\ timers[i].r = \text{"no"}$
$\land\ timers' = [timers \text{ EXCEPT } ![i] = [t \mapsto @.t,\ l \mapsto lim,\ r \mapsto @.r]]$
$\land \text{ UNCHANGED } \langle now \rangle$

$Start(i) \triangleq \land\ timers[i].r = \text{"no"}$
$\land\ timers[i].l > 0$
$\land\ time' = now$
$\land\ timers' = [timers \text{ EXCEPT } ![i] = [t \mapsto @.t,\ l \mapsto @.l,\ r \mapsto \text{"yes"}]]$
$\land \text{ UNCHANGED } \langle now \rangle$

$Timeout(i) \triangleq \land\ timers[i].r = \text{"yes"}$
$\land\ now - timers[i].t \geq timers[i].l$
$\land\ timers' = [timers \text{ EXCEPT } ![i] = [t \mapsto @.t,\ l \mapsto @.l,\ r \mapsto \text{"no"}]]$
$\land \text{ UNCHANGED } \langle now \rangle$

$Stop \triangleq \land\ timers[i].r = \text{"yes"}$
$\land\ timers' = [timers \text{ EXCEPT } ![i] = [t \mapsto @.t,\ l \mapsto @.l,\ r \mapsto \text{"no"}]]$
$\land \text{ UNCHANGED } \langle now \rangle$

$TNext \triangleq \exists\, i \in Nat : Start(i) \lor Stop(i) \lor Timeout(i) \lor (\exists\, t \in Nat : Set(i,\ t))$
$TSpec \triangleq TInit \land \Box[TNext]_{timers} \land (\forall\, i \in Nat : RTBound(Timeout(i), \langle timers,\ now \rangle, 0, 1))$

THEOREM $TSpec \Rightarrow \Box\, TTypeInv$